NeuroMIMO

Review Preparation Notes

Wireless Mobile Communication • Project Review

Pitch, talk script, numbers to memorise, theory refresher, expected questions with answers, and honest answers about known weaknesses.

October 30, 2026

Contents

1. How to Use These Notes Tonight 2
2. Your 60-Second Pitch 2
3. Five-Minute Review Talk 2
4. Project Numbers to Memorise 4
5. Wireless Mobile Communication Refresher 5
6. Expected Questions and Model Answers 8
7. Known Weaknesses: Answer These Honestly 11
8. Demo Plan and Checklist 12

1. How to Use These Notes Tonight

Your review is about two things: whether you understand **Wireless Mobile Communication** theory, and whether you understand **your own project** well enough to defend it. These notes cover both. Suggested plan for tonight (about 2½ hours):

Time	Do this	Section
20 min	Practise the 60-second pitch and the 5-minute talk out loud, twice.	2, 3
15 min	Memorise the project numbers card.	4
45 min	Read the theory refresher; write each formula once by hand.	5
40 min	Cover the answers and test yourself on the expected questions.	6
15 min	Read the weaknesses section carefully; these are the hardest questions.	7
15 min	Check the demo works and pack the backups.	8

Golden rules for the review. (1) Answer the question asked, then stop. (2) Use your own numbers: “at 0 dB the DNN gives −8.7 dB NMSE” sounds far stronger than “it works well”. (3) If you do not know, say so and say how you would find out; never invent. (4) Admit weaknesses before the professor finds them; it shows you understand your work.

2. Your 60-Second Pitch

Learn this almost word for word. It is the answer to “Tell me about your project.”

“My project, NeuroMIMO, studies **channel estimation** in a **2×2 MIMO-OFDM** system, the same physical-layer technology used in 4G, 5G and Wi-Fi. The receiver must know how the wireless channel distorts the signal, and it learns this from known **pilot** symbols. I compare three estimators on identical channels: **Least Squares**, **Linear MMSE**, and a **residual deep neural network** in PyTorch that refines the LS estimate. I simulate Rayleigh multipath fading over 0 to 30 dB SNR and measure NMSE, bit error rate with confidence intervals, and speed. The DNN slightly beats LS at low SNR and comes close to LMMSE without needing channel statistics, but it hits an error floor around −25 dB at high SNR, which I analyse. Everything runs through a FastAPI backend and a React dashboard, so results are reproducible and interactive.”

3. Five-Minute Review Talk

One idea per minute. Show the dashboard or the PDF figures while you speak.

Minute 1 — Problem

Wireless signals reach the receiver through many reflected paths (multipath), so each frequency is attenuated and rotated differently, and noise is added. In MIMO-OFDM the receiver must estimate this channel to separate the two streams and recover the data. Classical LS is simple but noisy; LMMSE is better but needs channel statistics and a matrix inverse. Can a neural network do better?

Minute 2 — System model

2 transmit and 2 receive antennas, 64 subcarriers, 16-sample cyclic prefix, 14 OFDM symbols per frame, QPSK. Comb pilots: antenna 0 on subcarriers 0, 16, 32, 48 and antenna 1 on 8, 24, 40, 56, so they never overlap. Channel: 4-tap Rayleigh with an exponential power delay profile, plus AWGN.

Minute 3 — Estimators

LS divides the received pilot by the known pilot and interpolates with a DFT method. LMMSE applies a Wiener filter using the channel correlation. The DNN is a residual MLP, 512→256→128→256→512, 330,112 parameters: it takes the LS estimate and learns only the correction. Training data are split by channel so the test channels are never seen.

Minute 4 — Results

Show the NMSE-vs-SNR figure. At 0 dB LMMSE is best (−9.1 dB), then the DNN (−8.7 dB) and LS (−8.4 dB). Above about 15 dB the DNN levels off near −25 dB while LS reaches −38.6 dB at 30 dB. All estimators run in under 1 ms per frame. BER points carry 95% Wilson confidence intervals.

Minute 5 — Insight and future work

The floor comes from training one small network over the whole SNR range. Fixes: give the SNR to the network as an input, use a CNN over time and frequency, and test realistic ray-traced channels, which I have started with a DeepMIMO loader. The software is tested with 88 automated tests.

4. Project Numbers to Memorise

Item	Value	Why it matters
Antennas	2 × 2 MIMO	Two parallel streams (spatial multiplexing)
Subcarriers / CP	64 / 16 samples	CP is 25% of the symbol, 20% overhead; must exceed the channel length
Frame	14 OFDM symbols	Same as one 5G NR slot
Pilots	4 per transmit antenna	Spacing 16 per antenna; enough to resolve 4 taps
Data subcarriers	56 per symbol	64 minus 8 pilot positions
Bits per frame	3,136 (QPSK)	2 antennas × 14 symbols × 56 × 2 bits
Channel	4-tap Rayleigh, exp. PDP	Frequency-selective, block fading
SNR range	0 to 30 dB, step 5	15 trials per point, 47,040 bits per point
DNN	512 → 256 → 128 → 256 → 512	Residual MLP, LayerNorm, GELU, dropout 0.05
DNN parameters	330,112	Small enough for real-time CPU inference
Training	30,000 samples, 40 epochs	Adam, MSE loss, 70/15/15 split, best-validation checkpoint
Equaliser	MMSE < 15 dB, ZF ≥ 15 dB	MMSE avoids noise boost at low SNR
Tests	88 automated tests	pytest; frontend builds with TypeScript

Key Results

Metric	LS	LMMSE	DNN
NMSE @ 0 dB	−8.38 dB	−9.05 dB	−8.67 dB
NMSE @ 15 dB	−23.09 dB	−23.19 dB	−19.56 dB
NMSE @ 30 dB	−38.64 dB	−38.64 dB	−25.96 dB
BER @ 30 dB	0.0018	0.0018	0.0043
Mean latency	0.12 ms	0.40 ms	0.65 ms

> **How to read NMSE in dB:** −10 dB means the error power is 10% of the channel power; −20 dB means 1%; −30 dB means 0.1%. So −38.6 dB is about 0.014% error.

5. Wireless Mobile Communication Refresher

The theory most likely to be asked, each linked to where it appears in your project.

5.1 Large-Scale Propagation

- **Free-space (Friis):** received power falls with the square of distance.

$$P_r = P_t G_t G_r \lambda^2 / ((4\pi)^2 d^2)$$

- **Log-distance path loss** with path-loss exponent n (2 in free space, about 2.7–5 in cities) and log-normal **shadowing** X_σ from buildings and obstacles:

$$PL(d)\ [dB] = PL(d_0) + 10\, n \log_{10}(d / d_0) + X_\sigma$$

Project link: NeuroMIMO normalises average channel power to 1 and sets the SNR directly, so large-scale effects are absorbed into the SNR. DeepMIMO channels include real path loss, which the loader normalises away.

5.2 Small-Scale Fading and Multipath

- **Multipath:** copies of the signal arrive with different delays and phases and add up.
- **Rayleigh fading:** no line-of-sight path; many scattered paths make the channel a complex Gaussian, so its amplitude is Rayleigh distributed: $p(r) = (r/\sigma^2) \exp(-r^2/2\sigma^2)$.
- **Rician fading:** one strong line-of-sight path plus scatter; the K-factor is the ratio of LoS power to scattered power (K = 0 gives Rayleigh).
- **RMS delay spread σ_τ:** how spread out in time the echoes are.
- **Coherence bandwidth:** the frequency range over which the channel is roughly constant, $B_c \approx 1/(5\sigma_\tau)$ (50% correlation) or $1/(50\sigma_\tau)$ (90% correlation).
- **Flat vs frequency-selective:** if signal bandwidth < B_c the fading is flat; otherwise it is frequency-selective (different subcarriers fade differently).
- **Doppler shift:** $f_d = (v/\lambda)\cos\theta$; maximum $f_m = v/\lambda$.
- **Coherence time:** how long the channel stays constant, $T_c \approx 0.423 / f_m$.
- **Slow vs fast fading:** slow if the symbol time is much shorter than T_c; fast otherwise.

Project link: 4-tap channel = frequency-selective (the |H| plot rises and falls across subcarriers). Block fading (constant over the 14-symbol frame) = slow fading. Doppler is not modelled yet; it is listed as future work.

5.3 Noise, SNR and BER

- **AWGN:** additive white Gaussian noise from the receiver electronics.
- **SNR in dB** = $10 \log_{10}(P_{signal}/P_{noise})$. 3 dB = double, 10 dB = 10×, 30 dB = 1000×.
- **E_b/N_0** is energy per bit over noise density. For QPSK, $E_s/N_0 = 2\, E_b/N_0$ (2 bits per symbol).
- **QPSK BER in AWGN:** $P_b = Q(\sqrt{(2E_b/N_0)})$, falls exponentially (a "waterfall").
- **BER in Rayleigh fading:** $P_b = \tfrac{1}{2}(1 - \sqrt{(\gamma/(1+\gamma))}) \approx 1/(4\gamma)$ at high SNR, so it falls only linearly: fading costs a lot. Diversity (more antennas) makes it fall faster.
- **Shannon capacity:** $C = B \log_2(1 + SNR)$ bits/s.

5.4 Digital Modulation

- **QPSK:** 4 phases, 2 bits per symbol. **16-QAM:** 16 points, 4 bits per symbol, needs higher SNR for the same BER.
- **Gray coding:** neighbouring constellation points differ in one bit, so the most likely symbol error causes only one bit error.
- **Constellation diagram:** shows the received points; noise spreads them, a channel error rotates or scales them (see the Signal Visualization page).

5.5 OFDM

- Splits a wideband, frequency-selective channel into N narrow, flat subcarriers, so each needs only a one-tap equaliser H[k].
- Subcarriers are orthogonal: spacing $\Delta f = 1/T_{symbol}$, so their spectra overlap but do not interfere. Implemented with the IFFT (transmitter) and FFT (receiver).
- **Cyclic prefix:** copy the last samples to the front. If CP length ≥ channel length it removes inter-symbol interference and keeps subcarriers orthogonal (circular convolution).

$$Y[k] = H[k]\, X[k] + W[k]$$

- **Advantages:** simple equalisation, robust to multipath, flexible resource allocation (OFDMA).
- **Disadvantages:** high peak-to-average power ratio (PAPR); sensitive to frequency offset and Doppler (inter-carrier interference); CP overhead.
- **Standards:** LTE uses 15 kHz spacing; 5G NR uses $15 \cdot 2^{\mu}$ kHz (15, 30, 60, 120 kHz) with 14 symbols per slot.

Project link: 64 subcarriers, CP 16 ≥ 4 taps, so no ISI; 14 symbols per frame like a 5G NR slot.

5.6 MIMO

- **Diversity gain:** send the same data over several paths to fight fading (e.g. Alamouti code with 2 transmit antennas).
- **Multiplexing gain:** send different data streams at once; capacity grows by about $\min(N_t, N_r)$ times at high SNR.

$$C = \log_2 \det(I + (SNR/N_t)\, H\, H^H) \text{ bits/s/Hz}$$

- **Received signal per subcarrier:** $\mathbf{y} = \mathbf{H}\mathbf{x} + \mathbf{n}$, with $\mathbf{H}$ a 2×2 matrix.
- **Zero-forcing equaliser:** $\mathbf{W} = (\mathbf{H}^H\mathbf{H})^{-1}\mathbf{H}^H$; removes interference but amplifies noise.
- **MMSE equaliser:** $\mathbf{W} = (\mathbf{H}^H\mathbf{H} + \sigma^2\mathbf{I})^{-1}\mathbf{H}^H$; balances interference and noise.

Project link: 2×2 spatial multiplexing (two streams); MMSE equaliser below 15 dB, zero-forcing above.

5.7 Channel Estimation

- **Pilot patterns:** block-type (whole symbols of pilots; good for slow fading), comb-type (some subcarriers in every symbol; good for fast fading), lattice (both).
- **Sampling rule:** pilots sample the channel in frequency, so you need at least as many pilots as channel taps ($N_p \geq L$), i.e. spacing ≤ N/L. In time, pilots must repeat faster than the coherence time.
- **LS:** $\hat{H} = Y/X$ at pilots. Unbiased, no prior needed, but error = noise/$|X|^2$.

- **LMMSE:** $\hat{H} = \mathbf{R}_{HP}(\mathbf{R}_{PP} + \sigma^2\mathbf{I})^{-1}\hat{H}_{LS}$. Uses channel correlation; best linear estimator; needs statistics and an $O(N_p^3)$ inverse.
- **Interpolation:** linear, spline, or DFT-based (transform to time, keep the taps, transform back).
- **Deep learning:** learn the estimator from data (e.g. ChannelNet, DNN estimators).

Project link: comb pilots, 4 per antenna = 4 taps, exactly enough. With spacing 16 overall (2 pilots per antenna) every estimator fails, which demonstrates the sampling rule.

5.8 Cellular Basics (in case they come up)

Generation	Key technology	Multiple access
1G (1980s)	Analogue voice	FDMA
2G (GSM, 1990s)	Digital voice, SMS	TDMA + FDMA
3G (UMTS)	Mobile data	CDMA
4G (LTE)	All-IP broadband, MIMO-OFDM	OFDMA (downlink)
5G (NR)	Massive MIMO, mmWave, flexible numerology	OFDMA

- **Cell and frequency reuse:** the area is divided into cells; the same frequencies are reused in distant cells to increase capacity. Smaller cells mean more reuse.
- **Handoff (handover):** switching a call to a new base station as the user moves.
- **Co-channel interference:** interference from cells using the same frequency.

6. Expected Questions and Model Answers

Cover the answers with a sheet of paper and answer out loud first.

Basics

Q. What is channel estimation and why is it needed?

Working out how the channel changed the signal (gain and phase on each subcarrier, for each antenna pair). The receiver needs it to equalise and to separate the MIMO streams; wrong estimates mean bit errors.

Q. Why use OFDM?

It turns a frequency-selective channel into many flat subchannels, each equalised with one complex gain, and the cyclic prefix removes inter-symbol interference.

Q. Why is the cyclic prefix 16 samples?

It must be at least as long as the channel impulse response. The channel has 4 taps, so 16 gives a large safety margin. The cost is 20% overhead (16 of 80 samples).

Q. What is the difference between diversity and multiplexing?

Diversity sends the same information over several paths for reliability; multiplexing sends different streams at once for higher data rate. My project uses 2×2 spatial multiplexing.

Q. What is Rayleigh fading?

Fading with no line-of-sight path: many scattered paths add up to a complex Gaussian channel whose amplitude follows a Rayleigh distribution.

Q. What are pilots and why comb-type?

Known symbols on known subcarriers. Comb pilots appear in every OFDM symbol, so they track changes over time; they suit fast-changing channels. The two antennas use different subcarriers so their pilots do not interfere.

Project design

Q. Why 2×2 MIMO and 64 subcarriers?

It is the smallest real MIMO case and keeps simulation and training fast on a laptop, while still showing frequency selectivity and stream separation. Scaling to 4×4 or more is future work.

Q. Why a residual network?

The LS estimate is already close. The network only learns the correction $\hat{H}_{DNN} = \hat{H}_{LS} + f(\hat{H}_{LS})$, which is easier to learn, trains faster and starts from LS quality instead of from zero.

Q. What is the input and output of the DNN?

Input: the LS estimate, 2×2×64 complex values split into 512 real numbers (real and imaginary parts). Output: 512 numbers forming the refined channel.

Q. Where does the training data come from?

It is simulated: random Rayleigh channels, random SNR between 0 and 30 dB, pilots sent through the channel with noise. The noisy LS estimate is the input and the true channel is the target. This is standard in channel-estimation research because the true channel is known in simulation.

Q. How did you avoid data leakage and overfitting?

Each sample uses its own new channel and the samples are split 70/15/15 into train, validation and test, so test channels are never seen. The model saved is the one with the best validation loss, with dropout 0.05 and LayerNorm.

Q. Why compare on identical channels?

The random generator is reset with the same seed for each estimator, so they all see the same channels and noise. Differences then come only from the estimators, not from luck.

Q. What is NMSE?

Mean squared estimation error divided by channel power, in dB. Lower is better: −20 dB means the error is 1% of the channel power.

Q. Why Wilson confidence intervals and the rule of three?

BER from a finite number of bits is uncertain. The Wilson interval gives a reliable 95% range; when zero errors occur, the rule of three gives the upper bound $3/N_{bits}$ instead of claiming a BER of zero.

Results

Q. Does the DNN beat LS?

At low SNR it is slightly better than LS (−8.7 dB vs −8.4 dB at 0 dB) and close to LMMSE (−9.1 dB). At high SNR it is worse: it levels off around −25 dB while LS keeps improving to −38.6 dB. I report this honestly rather than hiding it.

Q. Why does the DNN have an error floor at high SNR?

One small network is trained over the whole 0–30 dB range, so it learns an average amount of denoising. At high SNR the LS estimate is already very accurate and the network's own approximation error dominates. Fixes: give SNR as an input, train separate models per SNR band, or use a larger model.

Q. Why does BER level off instead of going to zero at high SNR?

In Rayleigh fading some frames hit deep fades, so BER only falls about linearly with SNR. With 15 frames per point a few bad channels dominate the errors.

Q. Is it fast enough for real systems?

Mean latency is about 0.1 ms for LS, 0.4 ms for LMMSE and 0.65 ms for the DNN per frame on a CPU, all well below a 1 ms 5G slot. On a GPU or with batching the DNN would be faster.

Q. What happens with fewer pilots?

With 2 pilots per antenna the channel (4 taps) is under-sampled and every estimator fails (about −4 dB NMSE). This confirms the rule that the number of pilots must be at least the number of channel taps.

Critical and future work

Q. What is new in your project?

An honest, reproducible comparison: identical channels for all estimators, the full 0–30 dB range including where the DNN loses, confidence intervals on BER, latency and generalisation tests, and an interactive dashboard plus a DeepMIMO loader for realistic channels.

Q. Your data is synthetic. Is that a problem?

Simulation is the standard first step because the true channel is known exactly. To move toward real conditions I added a DeepMIMO loader for ray-traced channels from realistic scenes; testing on those is the next step.

Q. What are the limitations?

Small 2×2 system, block fading with no Doppler, perfect synchronisation, one modulation for training, and the DNN high-SNR floor. Each is listed as future work.

Q. What would you do next?

Feed SNR to the network; try a CNN over time and frequency (like ChannelNet); add Doppler with the Jakes model; scale to 4×4 and 8×8; evaluate on DeepMIMO channels; learn pilot placement.

7. Known Weaknesses: Answer These Honestly

A good examiner looks for the weak spots. Knowing them and explaining them is a strength.

7.1 The LMMSE bug you found and fixed

What happened: earlier results showed LMMSE *worse* than LS at low SNR (−4.0 dB against −8.4 dB at 0 dB), which contradicts theory. If the professor has seen older slides or PDFs, be ready to explain.

The cause: the receiver averages each pilot over the 14 OFDM symbols of the frame, which reduces the pilot noise 14 times (about 11.5 dB). The LMMSE filter was given the un-averaged noise variance σ^2 instead of $\sigma^2/14$, so it assumed far more noise than there was and over-smoothed the estimate. **The fix** passes $\sigma^2/14$ to the estimators (the equaliser still uses σ^2, because data symbols are not averaged), with a regression test.

SNR	LS	LMMSE before fix	LMMSE after fix
0 dB	−8.38 dB	−4.01 dB	−9.05 dB
10 dB	−18.24 dB	−14.71 dB	−18.44 dB
20 dB	−28.95 dB	−27.30 dB	−28.93 dB
30 dB	−38.64 dB	−38.41 dB	−38.64 dB

Saved results, 15 trials per SNR point, seed 42.

> **What to say:** "I found that my LMMSE used the noise variance before pilot averaging over the 14 symbols. With the correct value $\sigma^2/14$ it is now the best estimator at low SNR, −9.05 dB against −8.38 dB for LS at 0 dB, as theory predicts. The gain is small because with 4 pilots and 4 taps the DFT-based LS estimate is already efficient."

7.2 The DNN model was retrained

The model file in the repository had been overwritten by a 2-epoch test model (an old test bug, since fixed). It was retrained on 30,000 simulated channels (21,000 training, 4,500 validation, 4,500 test) for 40 epochs with batch size 128 and learning rate 0.001, keeping the checkpoint with the best validation loss. All results and figures were then regenerated.

7.3 Other weaknesses

Weakness	Honest answer
DNN error floor at high SNR	Explained in Section 6; SNR-aware input or per-SNR models would fix it.
Only simulated channels	Standard practice; DeepMIMO loader added for realistic ray-traced channels.
No Doppler / mobility	Block fading assumed; Jakes Doppler model is planned.
Small 2×2 system	Chosen for speed and clarity; code is structured to scale.
Perfect synchronisation	Timing and frequency offset are not modelled, as in most estimation studies.
Only 15 trials per SNR	Confidence intervals are shown; more trials would narrow them.

8. Demo Plan and Checklist

Demo order (about 3 minutes)

- **Overview page:** point to the pipeline steps and the status cards (API connected, model ready).
- **MIMO-OFDM Simulator:** run at 0 dB, then at 20 dB, and show NMSE and BER improve.
- **Signal Visualization:** show the constellation and the channel frequency response (frequency selectivity).
- **Performance Comparison:** show the NMSE and BER curves and the summary cards.
- **ML Model Training:** show the loss curve (do not start a long training run during the review).

Start the app before you walk in

```
# Terminal 1 (project folder)
uvicorn neuromimo.api.main:app --app-dir backend --port 8000
# Terminal 2
cd frontend
npm run dev          # open http://localhost:5173
```

The first simulation after starting can take about 2 seconds while the model loads; run one simulation in advance so the demo is fast.

Checklist

- Laptop charged; charger packed; app already running; browser zoom set so charts are readable.
- Backup if the laptop fails: the PDFs in `docs/` and the figures in `backend/results/` on a USB drive or phone.
- Printed or offline copy of the 60-second pitch and the numbers card.
- Know where each part of the code is (Section 4 of the explainer PDF).
- Sleep. A rested explanation beats an extra hour of reading.